Day 30: Final Practical SOC Investigation

INC-20261002-001: Unauthorized-Style Local Administrator Account svc_update

SOC Internship - TriosCyber

Field	Details
Intern Name	Amrita Pathak
Date	2 October 2026
Internship Track	SOC (Security Operations Center)
Assigned Task	Perform an end-to-end investigation using a provided or self-generated safe lab scenario: collect telemetry, identify the alert/pattern, triage it, investigate logs, collect evidence, map relevant ATT&CK behavior and produce a closure or escalation recommendation. (Tools: Wazuh, Windows/Linux logs, MITRE ATT&CK)
Status of this Report	**Practical exercise, fully completed on 2 October 2026 on the DESKTOP-AUHDFVM lab VM. A self-generated safe scenario (creation of a local account and its addition to the Administrators group) was simulated, detected, triaged, investigated, contained and closed, with all evidence exported and SHA-256 sealed as case INC-20261002-001. Every screenshot is from this exercise. No Wazuh manager is deployed in this lab (see Days 17-20), so detection was performed from Windows Security logs; the Wazuh section describes the relevant rules from Wazuh's own published material and was not run live.**

Table of Contents

Table of Contents 2
Key Terms 4
Important Note on This Report 4
Executive Summary 5
1. Objective 6
2. Scenario and Workflow 7
3. Phase 1: Preparation and Telemetry Check 8
4. Phase 2: Simulation 10
5. Phase 3: Detection and Triage 12
5.1 Triage Decision 15
6. Phase 4: Investigation 17
6.1 Exporting the Logs 17
6.2 What Else Happened in the Window? 17
6.3 Exactly What Was Changed, and by Whom? 18
6.4 Was the New Account Used, and Where Did the Session Come From? 20
6.5 Investigation Findings 22
7. Phase 5: Evidence Collection 23
8. Phase 6: MITRE ATT&CK Mapping 25
8.1 T1136.001 Create Account: Local Account 25
8.2 T1098.007 Additional Local or Domain Groups 25
8.3 Techniques Considered and Not Applied 25
8.4 The Wazuh Side 26
9. Phase 7: Containment 27
10. Sealing the Evidence 30
11. Timeline 32
12. Severity, Verdict and Recommendation 33
12.1 Severity 33
12.2 Verdict 33
12.3 Closure or Escalation Recommendation 33
13. Recommendations 35
13.1 Recommendations Summary 37
14. Handling Issues During the Investigation 38

15. Conclusion 39
References 39

Key Terms

- **Persistence:** The MITRE ATT&CK tactic covering techniques an adversary uses to keep access to a system across restarts, password changes and other interruptions. Creating an extra account is a classic example.
- **Privilege Escalation:** The tactic covering techniques used to gain higher-level permissions, such as adding an account to the Administrators group.
- **Backdoor Account:** An account an attacker creates so that they can return later, even if the access they originally used is closed.
- **Audit Policy:** The Windows settings that decide which kinds of activity are written to the Security log. If a subcategory is set to No Auditing, the corresponding events are never recorded.
- **Subject:** In Windows account-management events, the account that performed the action (here, who created the account or changed the group).
- **SID (Security Identifier):** The unique identifier Windows assigns to every account and group. Group membership events record the member by SID.
- **Logon ID:** A value identifying one logon session. Events sharing a Logon ID were performed within the same session, which lets an analyst trace an action back to the logon that started it.
- **Containment:** Actions that stop an incident from continuing or spreading, such as disabling or deleting a malicious account.
- **Benign True Positive:** A detection that fired correctly, because the activity really happened, but which investigation shows was authorized and not malicious.

Important Note on This Report

The activity investigated here was a controlled simulation performed by the analyst on her own lab VM, and the account created was removed at the end. The investigation is nevertheless conducted exactly as it would be for a genuine alert: severity is first assessed on what an analyst would know when the alert arrived, and the verdict is reached only from evidence. All times are India Standard Time (UTC+05:30) unless marked UTC.

Executive Summary

On 2 October 2026 at 12:01:54 IST, a new local account named svc_update was created on the Windows endpoint DESKTOP-AUHDFVM, and 56 seconds later it was added to the local Administrators group. A newly created account receiving administrator rights within a minute is a recognized attacker pattern for establishing a backdoor, so the activity was triaged as High severity.

Investigation of the exported Security log showed that every change was made by the existing administrator CyberAmy within a single elevated, interactive console session that began at 11:34:36, with no remote source. The new account never logged on. The behavior maps to MITRE ATT&CK T1136.001 (Create Account: Local Account) and T1098.007 (Account Manipulation: Additional Local or Domain Groups).

The account's state was captured, and it was then disabled at 12:59:41 and deleted at 12:59:55, 58 minutes after creation. The clean-up was confirmed in the Security log. Eleven evidence files were exported and SHA-256 hashed. Because the activity was an authorized simulation, the case is closed as a benign true positive with a final severity of Low and no escalation. The investigation also identified a real visibility gap: no process creation events were recorded, so the command that created the account cannot be seen in the logs.

1. Objective

The objective of this final lab is to perform a complete SOC investigation from start to finish on a safe, self-generated scenario, bringing together every skill practiced during the month: confirming telemetry, identifying a suspicious pattern, triaging it, investigating the logs, collecting and sealing evidence, mapping the behavior to MITRE ATT&CK, and producing a justified closure or escalation recommendation.

To do this, a realistic but harmless scenario was chosen: creating a local account with a service-style name and adding it to the Administrators group on the lab VM. This was selected for three reasons. It is behavior real attackers use to keep access to a compromised machine. It is completely safe and reversible on a lab VM, since the account can simply be deleted. And it is different from the brute-force case investigated in Days 25 to 29, so it demonstrates that the investigation method can be repeated on a new type of activity and a different ATT&CK tactic.

The work was carried out in seven phases: preparation, simulation, detection and triage, investigation, evidence collection, ATT&CK mapping, and containment and closure. Each phase below explains what was done, why it matters in a SOC, and what the evidence showed.

2. Scenario and Workflow

In a real SOC, this investigation would begin with an alert such as "new local account added to Administrators". Here the analyst both generated the activity and investigated it, so the phases below separate the two roles clearly.

Phase	Purpose	Outcome
1. Preparation	Create the case folder; confirm the required logging is enabled	Auditing confirmed before any activity
2. Simulation	Generate the activity safely	Account created and made administrator
3. Detection and triage	Find the events; decide initial severity	7 account events found; triaged High
4. Investigation	Establish who, what, when, from where, and whether the account was used	One actor, one console session, account never used
5. Evidence collection	Export logs and capture system state	11 evidence files
6. ATT&CK mapping	Describe the behavior in standard terms	T1136.001 and T1098.007
7. Containment and closure	Remove the account; verify; seal; decide	Account deleted; case closed, Low

3. Phase 1: Preparation and Telemetry Check

Telemetry is the data a SOC relies on to see what happened: logs, alerts and system records. Before depending on any log source, an analyst confirms that it is actually being collected. Missing or disabled logging is one of the most common reasons real investigations fail, because events that were never recorded cannot be recovered afterwards.

Step 1: Create the case folder

Every investigation needs one place to hold its evidence, so the first action is to create a case folder. The folder is named INC-20261002-001 (incident, date, sequence number), following the same convention as the Day 26 case, so cases sort by date and never collide. The path is stored in a variable, $case, so later commands can refer to it without retyping it. Numbered subfolders separate case notes, raw logs, parsed logs and endpoint information, which keeps primary evidence apart from working copies.

```
$case = "C:\Incidents\INC-20261002-001"
"00_Case_Notes","01_Screenshots","02_Logs_Raw","03_Logs_Parsed",
"04_Endpoint_Info" | ForEach-Object {
    New-Item -ItemType Directory -Path "$case\$_" -Force }
```

Step 2: Confirm that account management auditing is enabled

Windows only writes an event to the Security log if the matching audit subcategory is switched on. This scenario depends on two of them: User Account Management, which records accounts being created, changed and deleted, and Security Group Management, which records members being added to or removed from groups. The auditpol command below displays the current setting of every subcategory in the Account Management category. It was run before the simulation, because if either subcategory were off, the events would never be created and could not be recovered afterwards.

```
auditpol /get /category:"Account Management"
```

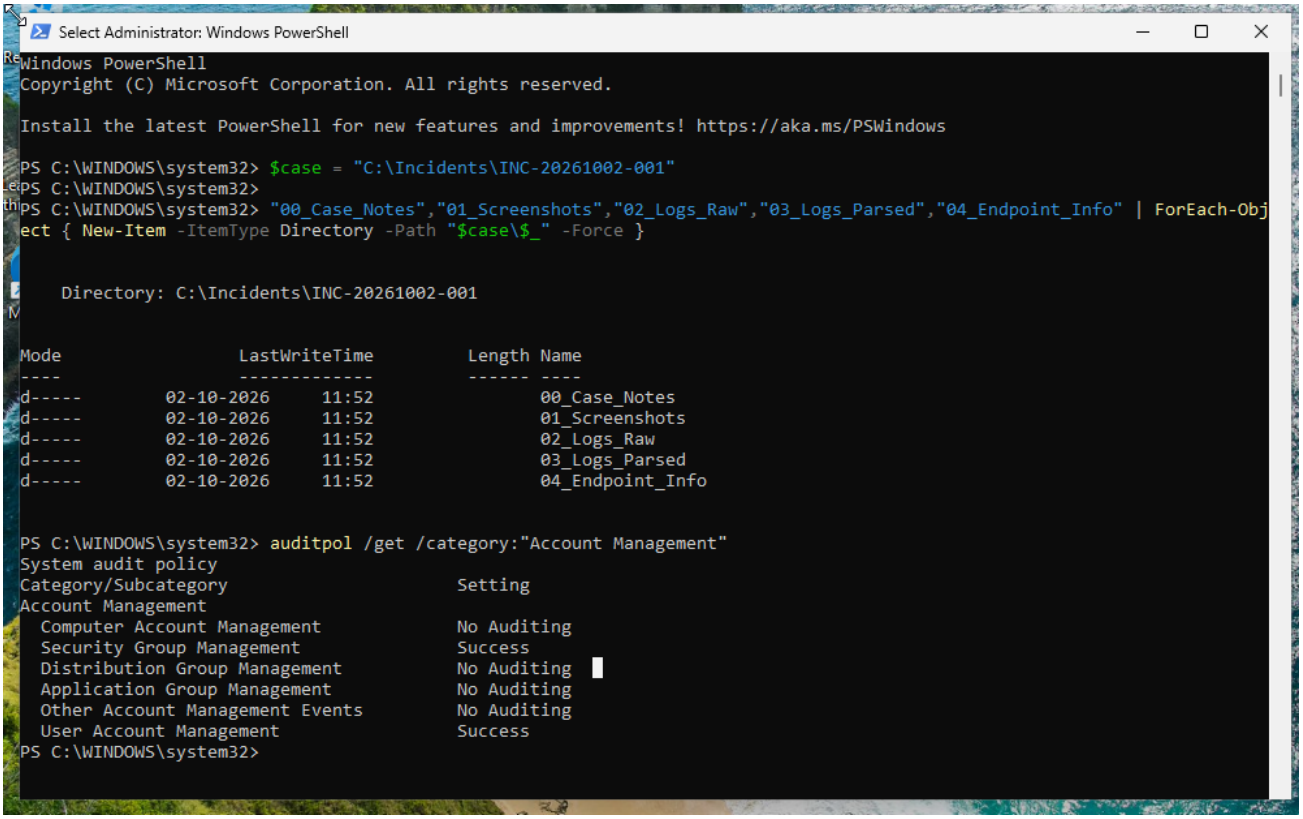


Figure 1. Case folder creation and the Account Management audit policy.

What this screenshot shows:

- The first two commands create the case folder INC-20261002-001 and its subfolders; the listing shows them created at 11:52 on 02-10-2026.
- auditpol /get lists the audit setting for each Account Management subcategory.
- User Account Management is set to Success. Microsoft lists events 4720 (account created), 4722, 4724, 4725, 4726 and 4738 under this subcategory, so account creation and deletion will be recorded.
- Security Group Management is set to Success. Microsoft lists events 4728, 4729, 4732 and 4733 under this subcategory, so group membership changes will be recorded.
- The other four subcategories show No Auditing. They are not needed for this scenario, but the output is recorded so the limits of the telemetry are known.

With both required subcategories confirmed, the simulation could proceed with confidence that its events would exist.

4. Phase 2: Simulation

The activity was generated with two built-in Windows commands, bracketed by Get-Date so the exact activity window is known.

Step 3: Create the account and add it to Administrators

Two commands reproduce what an attacker with administrator access would do to leave a backdoor: create a new account, then give it administrator rights. Get-Date is run immediately before and after, which records an exact start and end time for the activity. Knowing the activity window precisely makes it possible to separate the simulated events from everything else in the log.

```
Get-Date
net user svc_update * /add
net localgroup administrators svc_update /add
Get-Date
```

- net user svc_update * /add creates the local account. The * makes Windows prompt for the password, so it is never displayed or captured in a screenshot.
- net localgroup administrators svc_update /add adds the account to the local Administrators group, giving it full control of the machine.

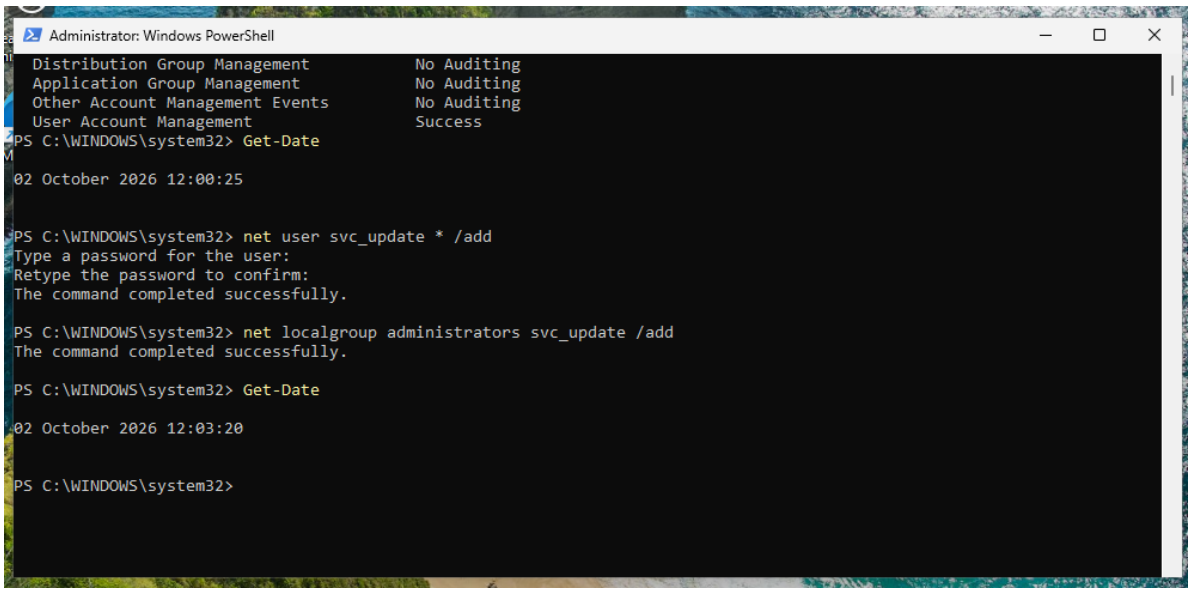


Figure 2. The simulated activity, with start and end times.

What this screenshot shows:

- Get-Date records the start of the activity window: 02 October 2026 12:00:25.
- net user svc_update * /add prompts twice for a password (nothing is shown while typing) and reports that the command completed successfully.
- net localgroup administrators svc_update /add also completes successfully: the new account is now an administrator.
- The second Get-Date records the end of the window: 12:03:20. All simulated activity therefore falls between 12:00:25 and 12:03:20 IST.

5. Phase 3: Detection and Triage

Detection is finding the events that indicate something happened. Triage is the first, fast assessment of those events: is this real, how serious could it be, and does it need full investigation? Triage does not aim to reach a final verdict; it decides priority.

Step 4: List recent account management events

The first task as an analyst is to find out whether the activity left a trace. The command below queries the live Security log for five event IDs associated with account creation and group changes, limited to the previous 30 minutes, and lists them in time order. It is a quick detection query, not the investigation itself: its purpose is to confirm that something happened and to show roughly what, so a triage decision can be made.

```
Get-WinEvent -FilterHashtable @{LogName='Security';
  Id=4720,4722,4724,4738,4732;
  StartTime=(Get-Date).AddMinutes(-30)} |
  Sort-Object TimeCreated |
  Format-Table TimeCreated, Id, TaskDisplayName -AutoSize
```

(Entered as a single line.) FilterHashtable selects the log, the event IDs and the start time; Sort-Object puts the results in time order; Format-Table displays the time, ID and category of each event.

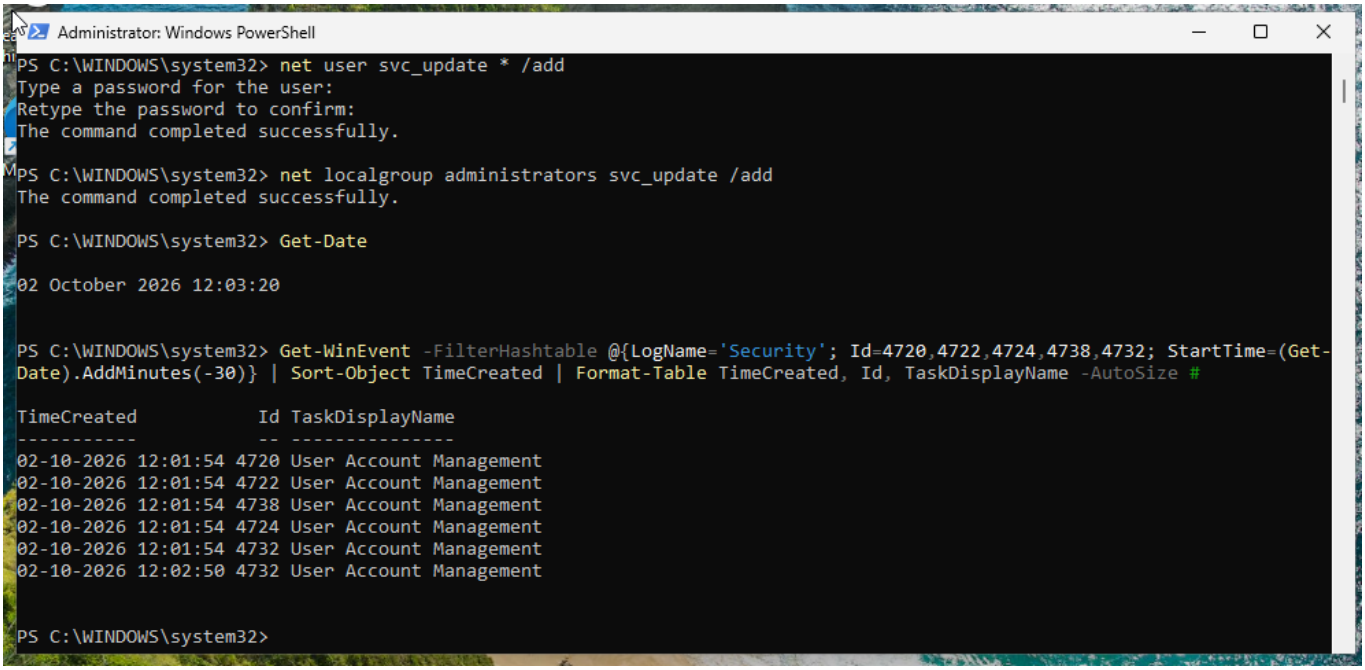


Figure 3. Account management events found in the Security log.

What this screenshot shows:

- Six events are returned. Five occur in the same second, 12:01:54: 4720 (a user account was created), 4722 (a user account was enabled), 4738 (a user account was changed), 4724 (an attempt was made to reset an account's password) and 4732 (a member was added to a security-enabled local group).
- A second 4732 occurs at 12:02:50, 56 seconds later.
- The TaskDisplayName column shows "User Account Management" for every row, including the 4732 events. Event Viewer shows 4732 under "Security Group Management" (Figure 5), which matches Microsoft's documentation. The reason for the difference in this column was not determined, so this report relies on the event IDs and Event Viewer rather than on that column.

Step 5: Open the key events in Event Viewer

A list of event IDs shows that something happened but not who did it or to what. The two most important events, the account creation (4720) and the later group change (4732), were therefore opened in Event Viewer (Windows Logs, Security) to read their details. For account management events, the Subject section identifies the account that performed the action, which is the key fact for triage.

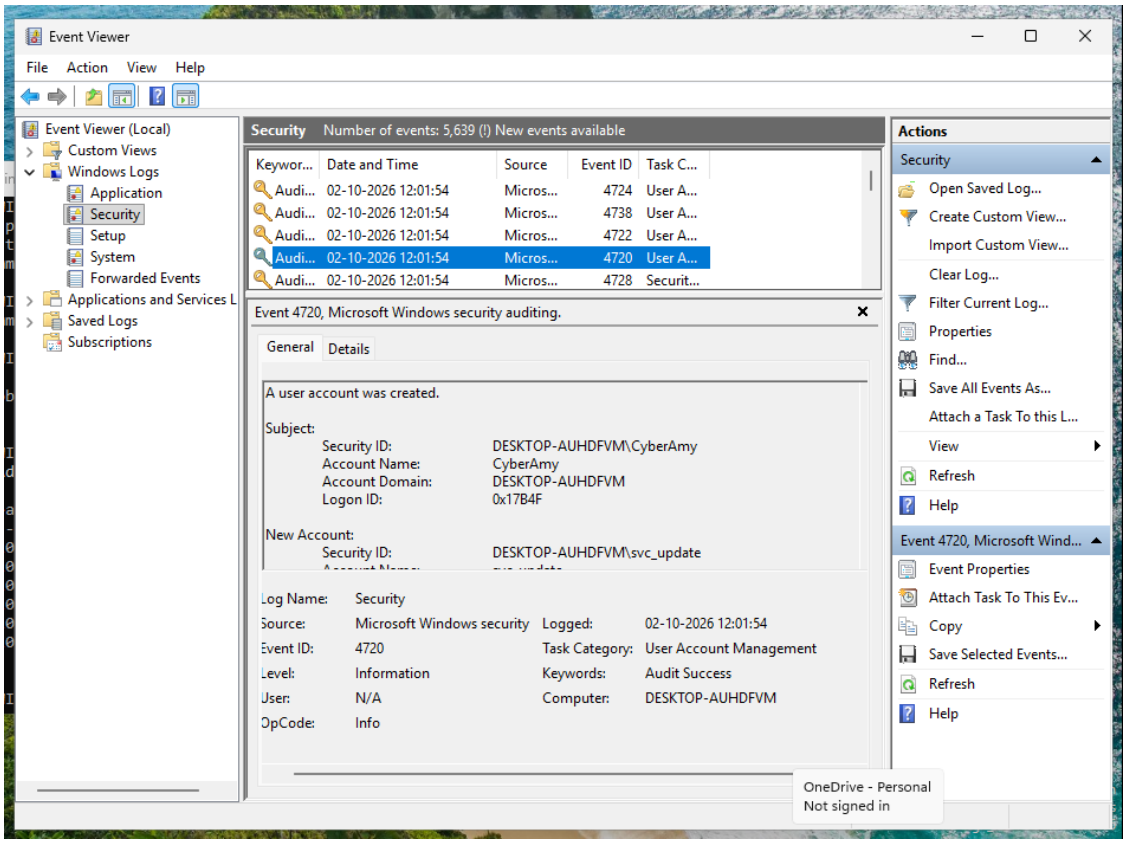


Figure 4. Event ID 4720, a user account was created.

What this screenshot shows:

- The event is logged at 02-10-2026 12:01:54 on computer DESKTOP-AUHDFVM, Task Category User Account Management.
- Subject is the account that performed the action: Security ID DESKTOP-AUHDFVM\CyberAmy, Account Name CyberAmy, Logon ID 0x17B4F.
- New Account shows the account that was created: DESKTOP-AUHDFVM\svc_update.
- In the event list above, an Event ID 4728 is visible at the same second. This ID was not in the filter used in Step 4, which is why the investigation in Section 6 exports every event in the window instead of only the expected IDs.

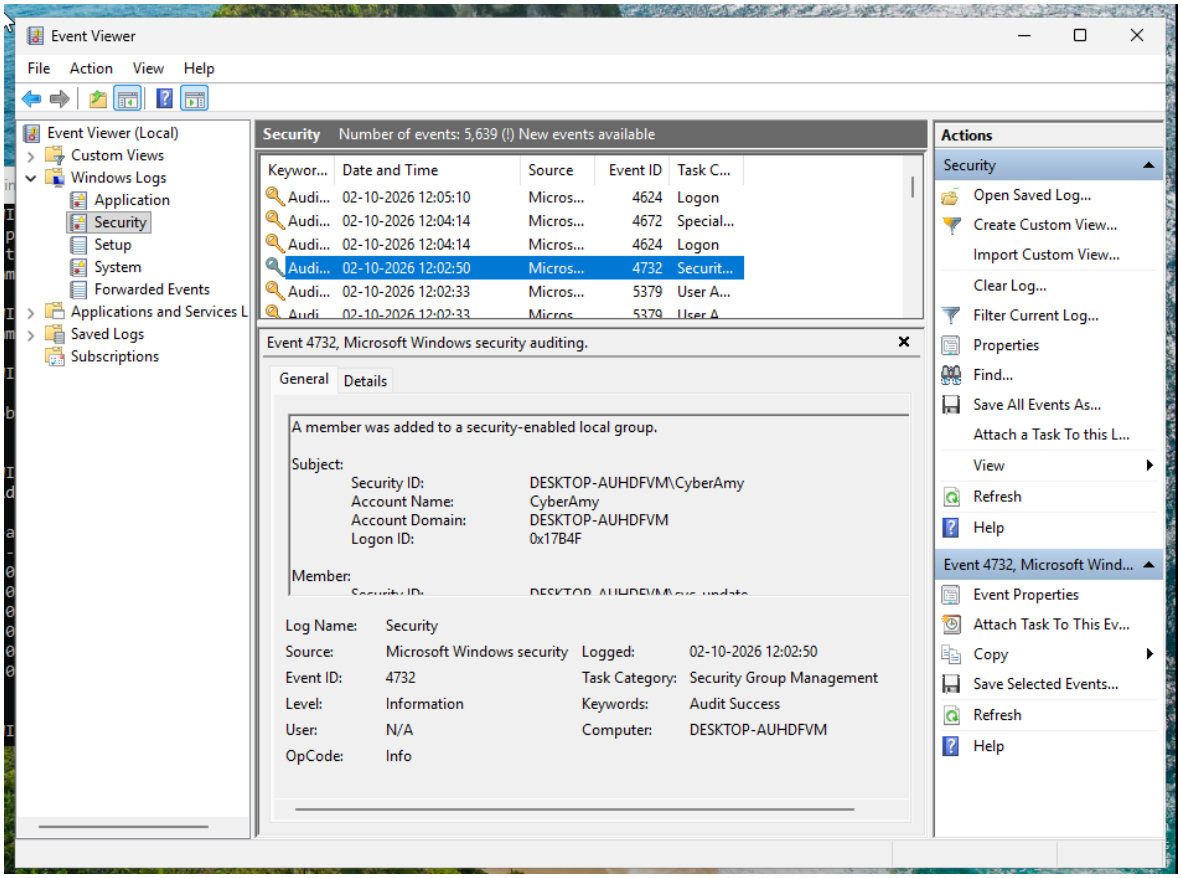


Figure 5. Event ID 4732 at 12:02:50, a member was added to a security-enabled local group.

What this screenshot shows:

- The event is logged at 12:02:50, Task Category Security Group Management.
- Subject is again CyberAmy with Logon ID 0x17B4F: the same account and the same logon session as the account creation.
- The Member section begins with a Security ID for DESKTOP-AUHDFVM\svc_update. The group name is below the visible area; it is confirmed as Administrators from the parsed evidence in Figure 7.

5.1 Triage Decision

Initial severity: High. The reasoning, based only on what is known at this point:

- A new account was given administrator rights within a minute of being created. Legitimate administrator accounts are rarely created this way without a change record.
- The account name svc_update looks like a service account, a naming style attackers use so an account blends in.
- Administrator rights mean full control of the endpoint, so the potential impact is high.

- The activity was performed by an existing administrator account, CyberAmy. If that account were compromised, this is exactly what an attacker would do next.

Triage outcome: investigate fully. The alert cannot be closed on the events alone.

6. Phase 4: Investigation

Investigation answers the questions triage raised. For this alert there are four: exactly what was changed, who changed it and from where, whether anything else happened in the same window, and whether the new account was ever used. Following the practice established in Day 26, the logs were exported first and all analysis was performed on the exported copy.

6.1 Exporting the Logs

Step 6: Export the incident window and the full Security log

The activity window (12:00 to 12:04 IST) is 06:30 to 06:34 UTC. The export covers 06:25 to 06:40 UTC (11:55 to 12:10 IST) and, unlike Step 4, includes every event in the window, not only expected IDs.

```
$q = "*[System[TimeCreated[
  @SystemTime>='2026-10-02T06:25:00.000Z' and
  @SystemTime<='2026-10-02T06:40:00.000Z']]]"
wevtutil epl Security "$case\02_Logs_Raw\Security_INC_window.evtx" "/q:$q"
wevtutil epl Security "$case\02_Logs_Raw\Security_full.evtx"
```

6.2 What Else Happened in the Window?

Step 7: Count every event type in the exported window

Triage looked only at the event IDs that were expected. An investigation must also look for what was not expected, so this step counts every event ID present in the exported window. Group-Object groups the events by ID and Sort-Object orders them by how often each occurs. The result shows the full picture of activity around the incident and reveals anything the earlier filter missed.

```
Get-WinEvent -Path "$case\02_Logs_Raw\Security_INC_window.evtx" |
  Group-Object Id | Sort-Object Count -Descending |
  Format-Table Name, Count -AutoSize
```

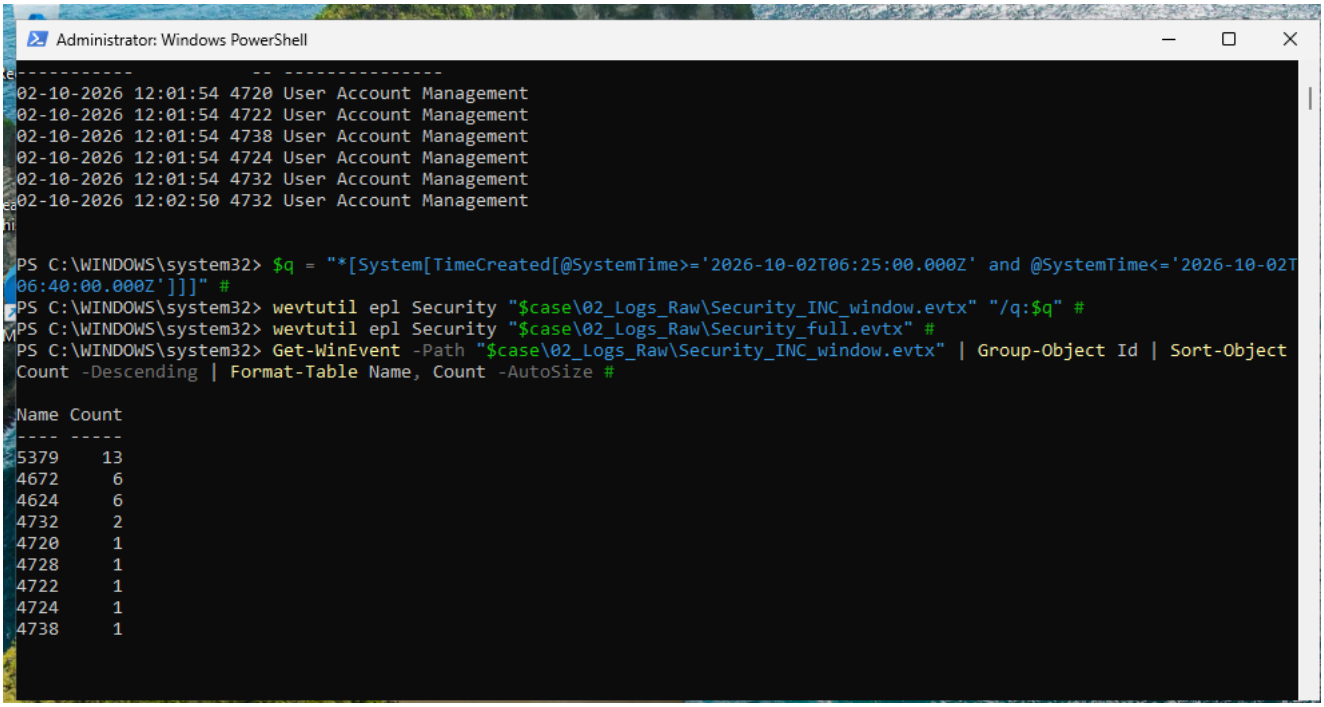


Figure 6. Log exports and the count of each event ID in the incident window.

What this screenshot shows:

- The $q line defines the time filter and the two wevtutil lines export the window and the full log. They print nothing, which for wevtutil means success.
- The table counts 32 events in the window: 5379 (13), 4672 (6), 4624 (6), 4732 (2), and one each of 4720, 4728, 4722, 4724 and 4738.
- The seven account and group events (4720, 4722, 4724, 4738, 4728 and two 4732) are the activity under investigation.
- 4624 is a successful logon and 4672 is special privileges assigned to a new logon. Six of each are present; Section 6.4 identifies them.
- 5379 is titled "Credential Manager credentials were read". The 13 occurrences were not examined individually in this investigation; they are preserved in the exported window for follow-up.
- No Event ID 4688 (process creation) appears, so the log does not record which program made these changes. This visibility gap is addressed in Section 13.

6.3 Exactly What Was Changed, and by Whom?

Step 8: Parse the account and group events from the exported file

This step extracts the details of each account and group event from the exported evidence file: when it happened, who performed it (the Subject), in which logon session, and what it affected. For group events, it also extracts the name of the group and the SID of the member added. The result is saved as a CSV in 03_Logs_Parsed and displayed as a table. This answers the question of exactly what was changed and whether one actor was responsible for all of it.

```
$f = "$case\02_Logs_Raw\Security_INC_window.evtx"
$ids = 4720,4722,4724,4738,4728,4732
function P($e){ $d=@{};
  ([xml]$e.ToXml()).Event.EventData.Data |
  % { $d[$_.Name]=$_.'#text' }; $d }
$rows = Get-WinEvent -Path $f | ? { $ids -contains $_.Id } |
  % { $d=P $_; [pscustomobject]@{
    Time=$_.TimeCreated.ToString("HH:mm:ss"); Id=$_.Id;
    Actor=$d.SubjectUserName; ActorLogonId=$d.SubjectLogonId;
    Target=$d.TargetUserName; MemberSid=$d.MemberSid} } |
  Sort-Object Time, Id
$rows | Export-Csv "$case\03_Logs_Parsed\account_events.csv" -NoTypeInformation
$rows | Format-Table -AutoSize
```

The function P reads an event's named data fields into a lookup table, so the later commands can stay short. % and ? are PowerShell shorthand for ForEach-Object and Where-Object. For group events, the Target column holds the group name.

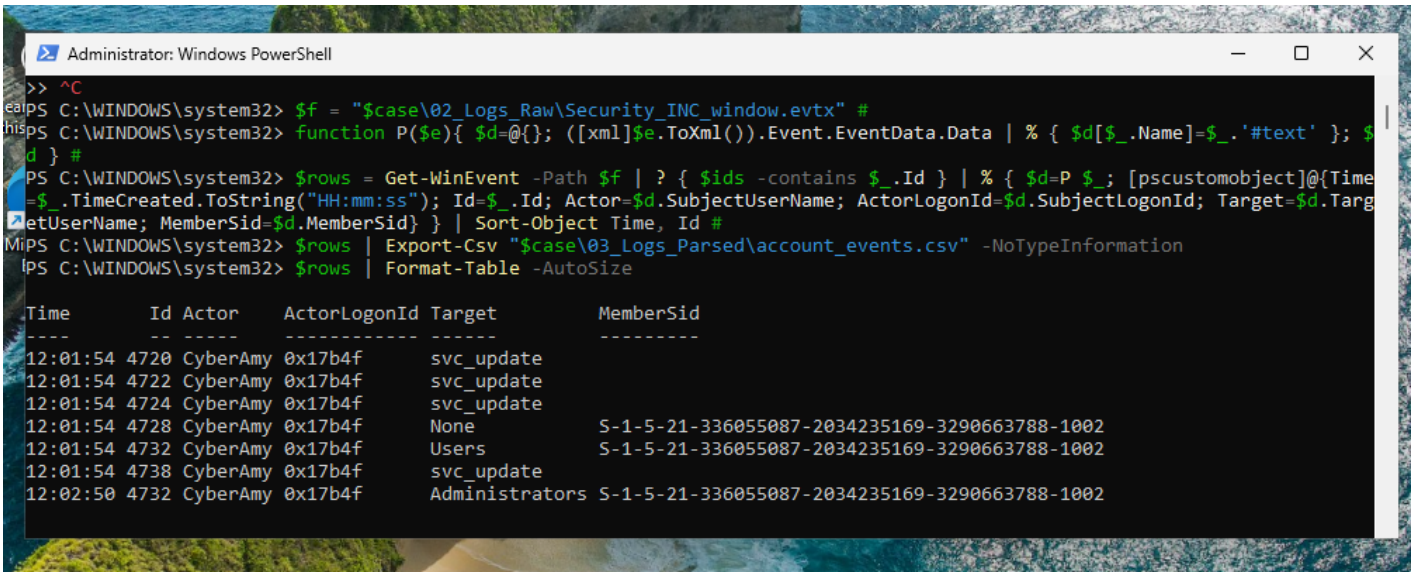


Figure 7. The seven account and group events, parsed from the exported evidence file.

What this screenshot shows:

- Every row has the same Actor, CyberAmy, and the same ActorLogonId, 0x17b4f: one account in one logon session made all seven changes.
- At 12:01:54, events 4720, 4722, 4724 and 4738 all have the target svc_update: the account was created, enabled, had its password set and was changed.
- Also at 12:01:54, event 4728 adds a member to the group None and event 4732 adds a member to the group Users. The net user output in Figure 9 lists None as the account's global group and Users as a local group, so these two memberships were assigned automatically when the account was created.
- At 12:02:50, event 4732 adds a member to Administrators. This is the only deliberate privilege change.
- The MemberSid is identical on all three group events and ends in -1002, confirming they all concern the same account, svc_update.

One reference source, Ultimate Windows Security, states that event 4728 is only logged on domain controllers. The evidence here shows it logged on a standalone workstation for the built-in group None. The report records what the evidence shows.

6.4 Was the New Account Used, and Where Did the Session Come From?

Step 9: List every logon in the window, then trace the actor's session

Two questions remain, and both affect the verdict. First, was the new account used? If svc_update had logged on, the incident would be far more serious, so every successful logon (Event ID 4624) in the window is listed with its account and logon type. Second, where did the actor's session come from? Every change carried Logon ID 0x17b4f, so the full exported log is searched for the logon event that created that session, which shows when it began, how the user logged on, and from what address.

```
Get-WinEvent -Path $f | ? { $_.Id -eq 4624 } | % { $d=P $_;
  [pscustomobject]@{Time=$_.TimeCreated.ToString("HH:mm:ss");
  User=$d.TargetUserName; LogonType=$d.LogonType} } |
  Format-Table -AutoSize
```

```
Get-WinEvent -Path "$case\02_Logs_Raw\Security_full.evtx" -FilterXPath
  "*[System[EventID=4624] and EventData[Data[
  @Name='TargetLogonId']='0x17b4f']]" | % { $d=P $_;
  [pscustomobject]@{Time=$_.TimeCreated; User=$d.TargetUserName;
  LogonType=$d.LogonType; SourceIP=$d.IpAddress;
  Elevated=$d.ElevatedToken} } | Format-List
```

The second command searches the full exported log for the logon event (4624) that created session 0x17b4f, the session that performed every change.

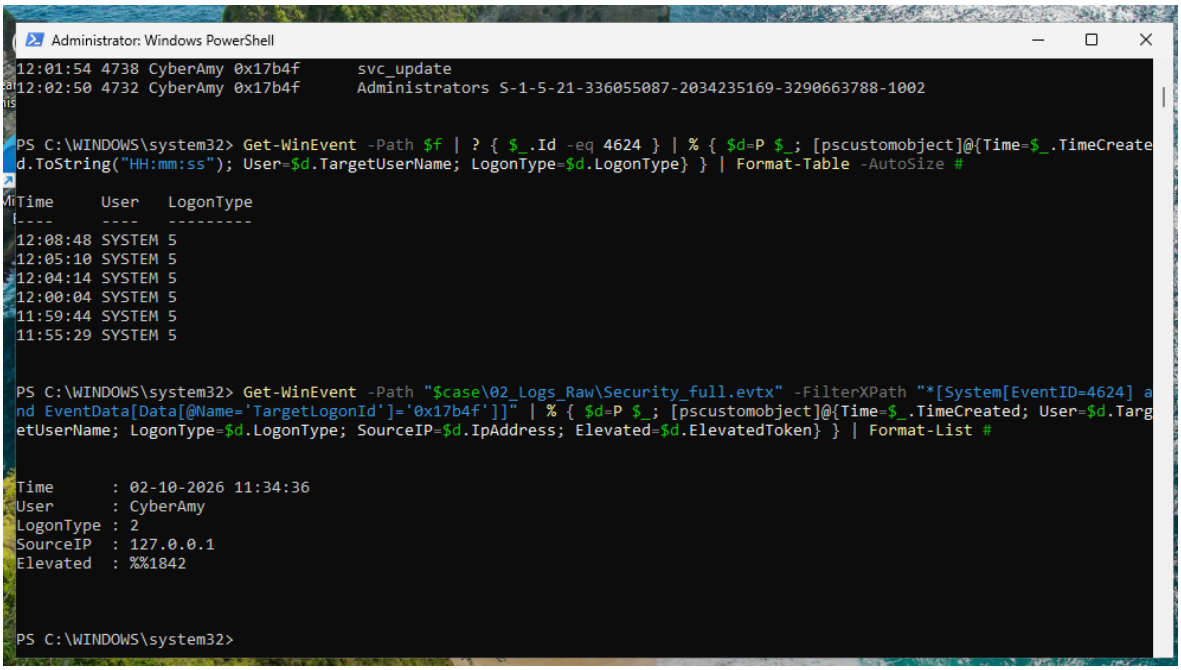


Figure 8. Logons in the incident window, and the origin of session 0x17b4f.

What this screenshot shows:

- The first table lists all six logons in the window (11:55:29 to 12:08:48). Every one is the account SYSTEM with Logon Type 5 (Service), which Microsoft defines as the Service Control Manager starting a service. None is svc_update: the new account did not log on. The six 4672 events in Figure 6 match these six SYSTEM logons in number.
- The second result shows the logon that created session 0x17b4f: user CyberAmy at 02-10-2026 11:34:36.
- LogonType 2 is an interactive logon, and SourceIP 127.0.0.1 means localhost: the session was started at the machine's own console, not over the network.

- Elevated %%1842 means Yes: this is the elevated administrator half of CyberAmy's logon, consistent with the commands being run in an administrator PowerShell window.

6.5 Investigation Findings

The purpose of an investigation is to replace the assumptions made at triage with facts. At triage, the activity looked like a possible backdoor created through a compromised administrator account. The investigation tested that reading against the exported evidence, and the findings below changed the assessment.

In summary: one known account, CyberAmy, made every change in a single session that began with an interactive logon at the machine's own console; nothing else of concern happened in the same window; and the new account was never used. The only question the logs could not answer was which program made the changes, because process creation events were not recorded. Each finding is listed with the evidence that supports it.

Question	Answer	Evidence
What was changed?	Account svc_update created; added to None and Users automatically, then to Administrators	Figure 7
Who did it?	CyberAmy, session 0x17b4f	Figures 4, 5, 7
From where?	Interactive console logon at 11:34:36 from 127.0.0.1, elevated	Figure 8
Anything else in the window?	Only SYSTEM service logons and 5379 events; no other account changes	Figures 6, 8
Was the new account used?	No. No logon by svc_update; Last logon Never	Figures 8, 9
Which program made the change?	Not recorded: no process creation (4688) events	Figure 6

7. Phase 5: Evidence Collection

Before any containment action changes the system, its current state is captured. Once an account is deleted, its properties (when its password was set, whether it ever logged on, which groups it belonged to) can no longer be queried, so this order matters.

Step 10: Capture the account, the administrators group and the system context

The commands below record the account's properties, the membership of the Administrators group, and the system context (system details, the collector's identity, the time zone and the collection time). Each output is saved into 04_Endpoint_Info. The two most important outputs are also shown on screen so they can be captured as screenshots.

```
net user svc_update | Tee-Object "$case\04_Endpoint_Info\net_user_svc_update_before.txt"
net localgroup administrators | Tee-Object "$case\04_Endpoint_Info\local_admins_before.txt"
systeminfo > "$case\04_Endpoint_Info\systeminfo.txt"
whoami /all > "$case\04_Endpoint_Info\collector_context.txt"
tzutil /g > "$case\04_Endpoint_Info\timezone.txt"
Get-Date -Format o > "$case\04_Endpoint_Info\collection_time.txt"
```

Tee-Object displays a command's output on screen and writes the same output to a file, so the screenshot and the evidence file contain identical information.

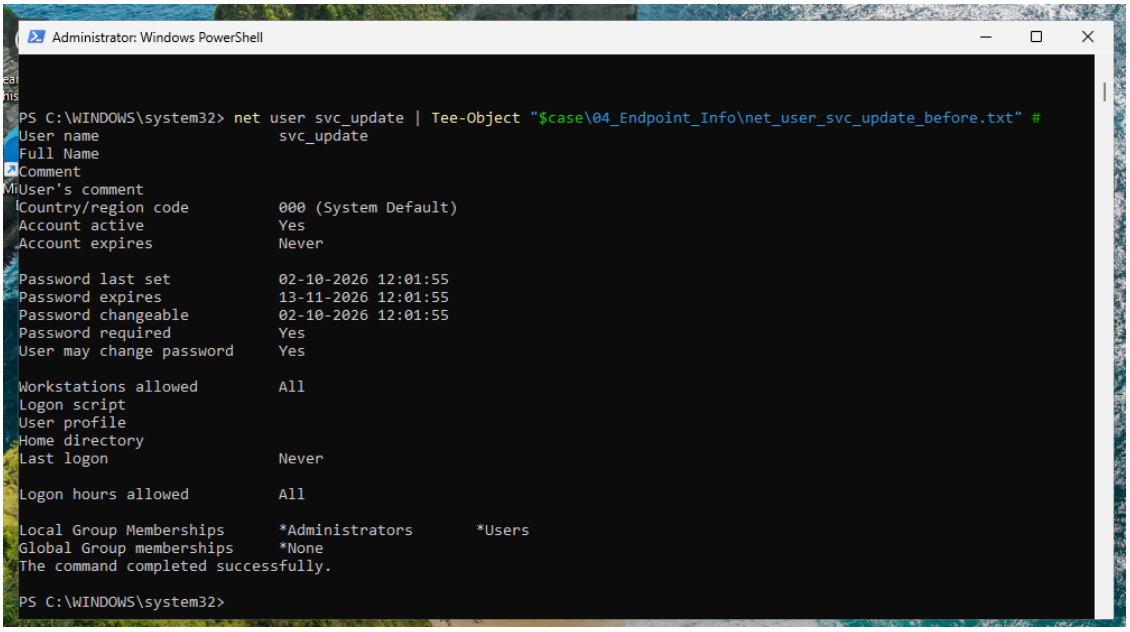


Figure 9. Properties of the svc_update account before containment.

What this screenshot shows:

- Account active: Yes. The account was enabled and usable.
- Password last set: 02-10-2026 12:01:55, one second after the creation events, consistent with the 4724 password event.
- Password expires: 13-11-2026, which is 42 days later, matching the maximum password age seen in the Day 25 net accounts output.
- Last logon: Never. This independently confirms the log finding that the account was not used.
- Local Group Memberships: Administrators and Users. Global Group memberships: None. These match the three group events in Figure 7 exactly.

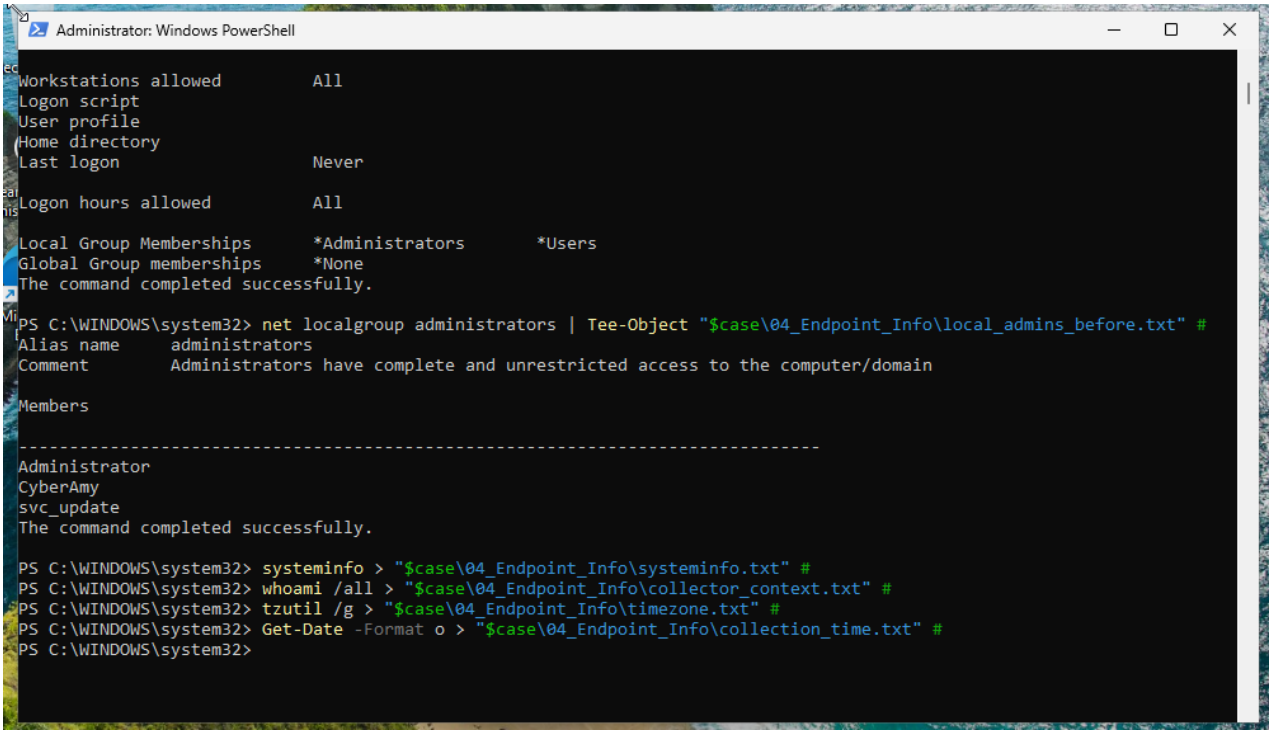


Figure 10. Members of the Administrators group before containment, and the remaining captures.

What this screenshot shows:

- The Administrators group has three members: Administrator, CyberAmy and svc_update. The new account is confirmed as an administrator.
- The four commands below it (systeminfo, whoami /all, tzutil /g and Get-Date -Format o) write the system, collector, time zone and collection time files. They print nothing because their output is redirected to files.

8. Phase 6: MITRE ATT&CK Mapping

Mapping describes observed behavior in the standard vocabulary used across the security industry. Both entries below were checked on attack.mitre.org on 2 October 2026.

Observed behavior	Technique	Tactic	Evidence
Local account svc_update created	T1136.001 Create Account: Local Account	Persistence (TA0003)	Event 4720, 12:01:54
Account added to Administrators	T1098.007 Account Manipulation: Additional Local or Domain Groups	Persistence (TA0003); Privilege Escalation (TA0004)	Event 4732, 12:02:50

8.1 T1136.001 Create Account: Local Account

MITRE's description begins: "Adversaries may create a local account to maintain access to victim systems." Its Windows detection analytic (AN1235, under detection strategy DET0447) describes an adversary using built-in tools like net user /add to create a local user, with account creation event 4720 following process creation of an executable such as net.exe. This matches the scenario: net user /add was used and event 4720 was recorded. The entry is version 1.5, last modified 12 May 2026.

8.2 T1098.007 Additional Local or Domain Groups

MITRE's description begins: "An adversary may add additional local or domain groups to an adversary-controlled account to maintain persistent access to a system or domain." Its Windows analytic (AN0865, under DET0310) detects unauthorized additions of users to privileged local groups such as Administrators. This matches the 4732 event for Administrators. The entry is version 1.1, last modified 26 September 2025, and belongs to two tactics, Persistence and Privilege Escalation.

8.3 Techniques Considered and Not Applied

- T1078 Valid Accounts: not applied. It covers an adversary logging on with an existing account's credentials. The new account never logged on, and CyberAmy's session was the legitimate owner's.
- T1036 Masquerading: noted but not mapped. The service-style name svc_update is the kind of name used to blend in, but a name alone is weak evidence of the technique.

This case used a different tactic from Day 27. That case was Credential Access (trying to obtain a password); this one is Persistence and Privilege Escalation (securing and widening access already held).

8.4 The Wazuh Side

No Wazuh manager is deployed in this lab, so no live alert exists. From Wazuh's own published material, its default Windows ruleset includes rule 60109, which fires when a user account is enabled or created, and rule 60110, when a user account is changed. A report in Wazuh's issue tracker describes rules 60144 and 60145 for members added to or removed from a security-enabled local group, and an aggregate rule 60154 (level 12, "Administrators Group Changed"), and states that the aggregate rule was not firing in that user's environment.

The practical consequence: account creation would be expected to alert by default, but an alert specifically for additions to Administrators should be tested, not assumed. This is carried forward as a recommendation.

9. Phase 7: Containment

Containment stops the threat from being used. For a suspicious account, the standard order is to disable it first, which blocks its use immediately while keeping it available for investigation, and to delete it only once its state has been recorded. Both steps were performed here so the logs show the full sequence.

Step 11: **Disable, then delete, the account**

Disabling the account (/active:no) immediately prevents anyone from logging on with it. Deleting it (/delete) then removes it from the system and from every group it belonged to. Get-Date is run before and after to record the containment window.

```
Get-Date
net user svc_update /active:no
net user svc_update /delete
Get-Date
```

Step 12: **Verify the removal**

A containment action is not complete until it has been verified. Two checks were made: listing the Administrators group again, to confirm the account is no longer a member, and querying the account itself, which should now fail because it no longer exists. The group listing is also saved as an after-containment evidence file.

```
net localgroup administrators | Tee-Object "$case\04_Endpoint_Info\local_admins_after.txt"
net user svc_update
```

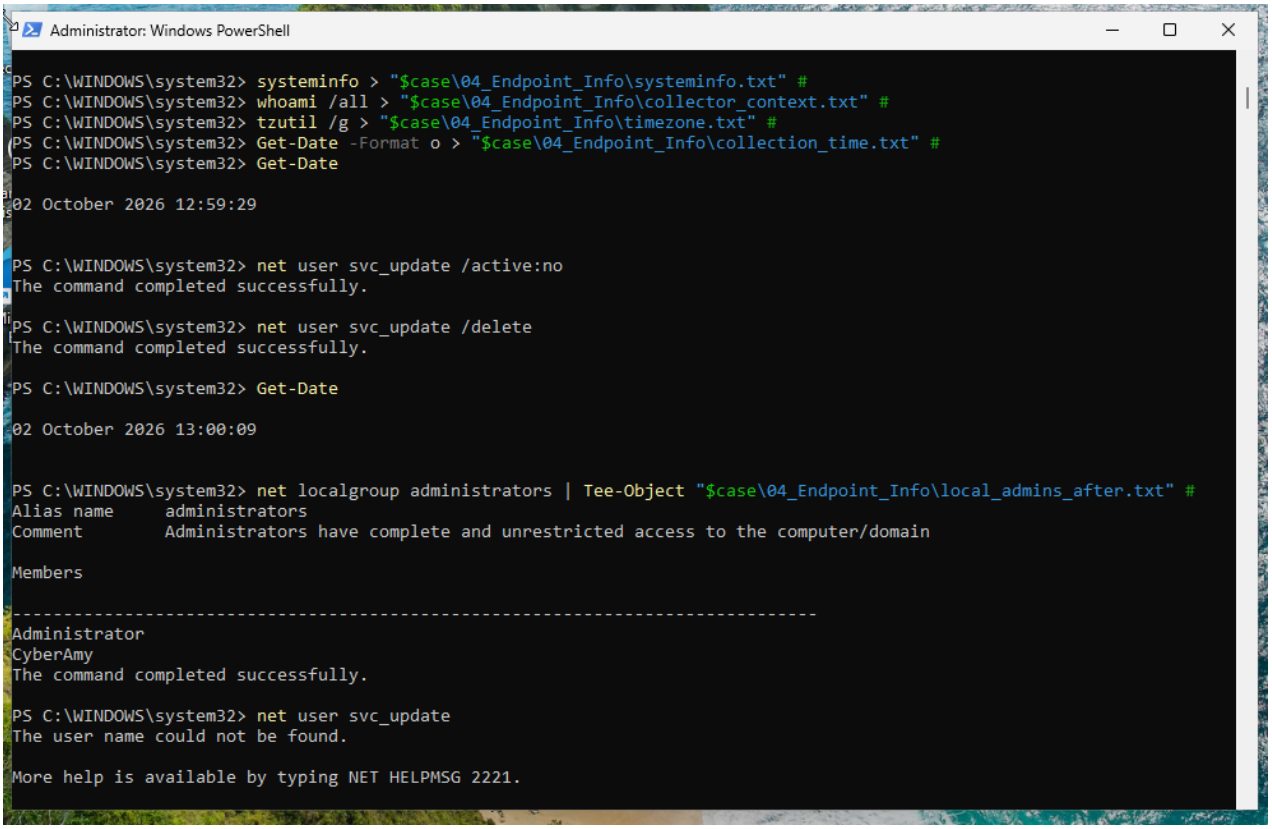


Figure 11. Containment and verification.

What this screenshot shows:

- Get-Date records the start of containment: 12:59:29.
- net user svc_update /active:no disables the account, and net user svc_update /delete removes it; both complete successfully. The second Get-Date shows 13:00:09.
- The Administrators group now lists only Administrator and CyberAmy: svc_update is no longer a member.
- net user svc_update returns "The user name could not be found." This error is the proof that the account no longer exists.

Step 13: Confirm the clean-up was logged, and export the log again

Removing an account also generates Security events, and confirming them serves two purposes: it proves the removal happened as intended, and it shows that an attacker removing a backdoor account to cover their tracks would be visible in the same way. The query below lists the disable, group-removal and delete events from the previous 10 minutes. The Security log is then exported a third time so these events are included in the evidence.

```
Get-WinEvent -FilterHashtable @{LogName='Security';
  Id=4725,4726,4729,4733;
  StartTime=(Get-Date).AddMinutes(-10)} |
  Sort-Object TimeCreated | Format-Table TimeCreated, Id -AutoSize
wevtutil epl Security "$case\02_Logs_Raw\Security_after_containment.evtx"
```

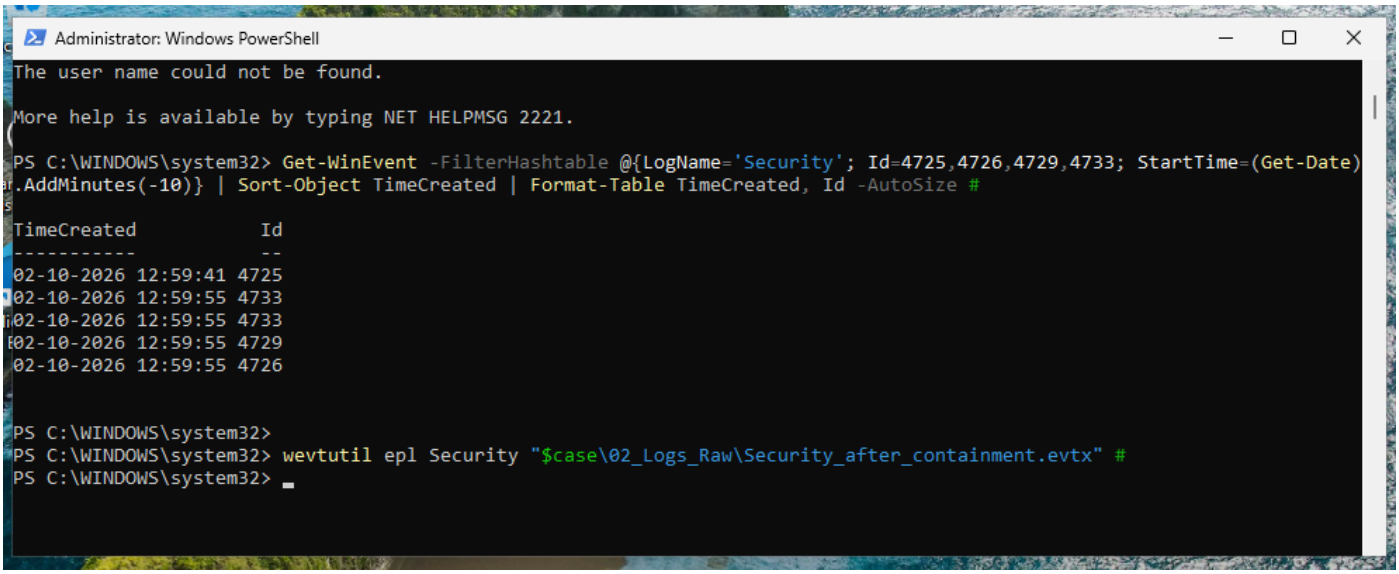


Figure 12. Clean-up events in the Security log.

What this screenshot shows:

- 12:59:41, event 4725: a user account was disabled.
- 12:59:55, two events 4733: a member was removed from a security-enabled local group. Two removals are consistent with the account's two local groups, Administrators and Users. The group names were not parsed individually; they are preserved in the exported log.
- 12:59:55, event 4729: a member was removed from a security-enabled global group, consistent with the group None.
- 12:59:55, event 4726: a user account was deleted.
- The final wevtutil line exports the Security log again, so the containment events are part of the sealed evidence.

The account existed from 12:01:54 to 12:59:55, a total of 58 minutes, and held administrator rights from 12:02:50 until it was disabled at 12:59:41.

10. Sealing the Evidence

Case notes (a case summary and a chain of custody record) were written and placed in 00_Case_Notes. Every evidence file was then hashed with SHA-256, so that any later change to a file can be detected.

Step 14: **Hash every evidence file and list the case**

The command below calculates a SHA-256 hash for every file in the case except the case notes, and writes the results to hash_manifest.csv. A hash is a fingerprint of a file's exact contents, so re-calculating it later and getting the same value proves the file has not changed. Hashing is done last, because any file modified after hashing would no longer match its recorded value.

```
Get-ChildItem $case -Recurse -File |
  ? { $_.FullName -notlike "*\00_Case_Notes\*" } |
  Get-FileHash -Algorithm SHA256 |
  Select-Object Hash, @{n='Item';e={$_.Path.Replace("$case\","")}} |
  Export-Csv "$case\00_Case_Notes\hash_manifest.csv" -NoTypeInformation
```

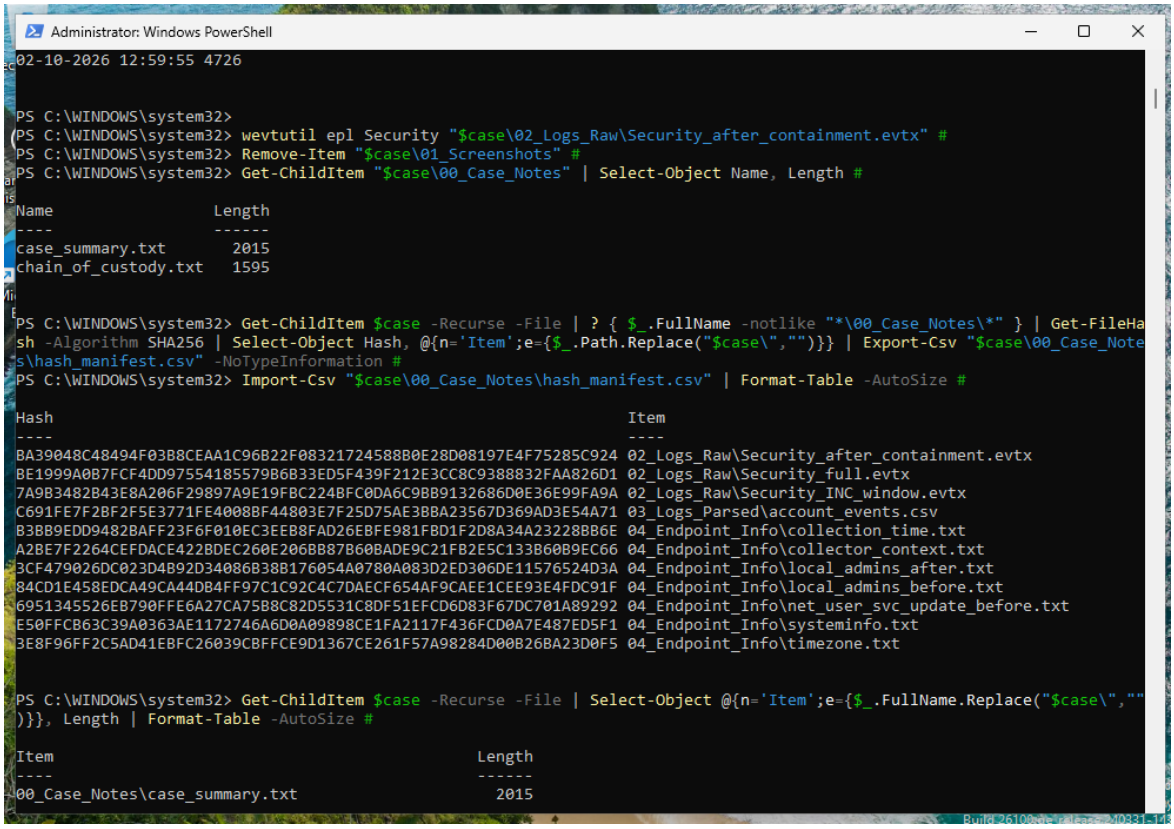


Figure 13. Case notes verified, and the SHA-256 hash manifest.

What this screenshot shows:

- Remove-Item deletes the unused 01_Screenshots folder. Screenshots were taken on the host laptop and are embedded in this report, which the chain of custody records.
- The listing of 00_Case_Notes shows case_summary.txt at 2015 bytes and chain_of_custody.txt at 1595 bytes, matching the sizes of the files as written, so they were transferred intact.
- The hash command excludes 00_Case_Notes, because notes may still be edited and a manifest cannot contain its own hash.
- The manifest lists 11 evidence files, each with a 64-character SHA-256 value: three exported logs, the parsed CSV and seven endpoint information files.

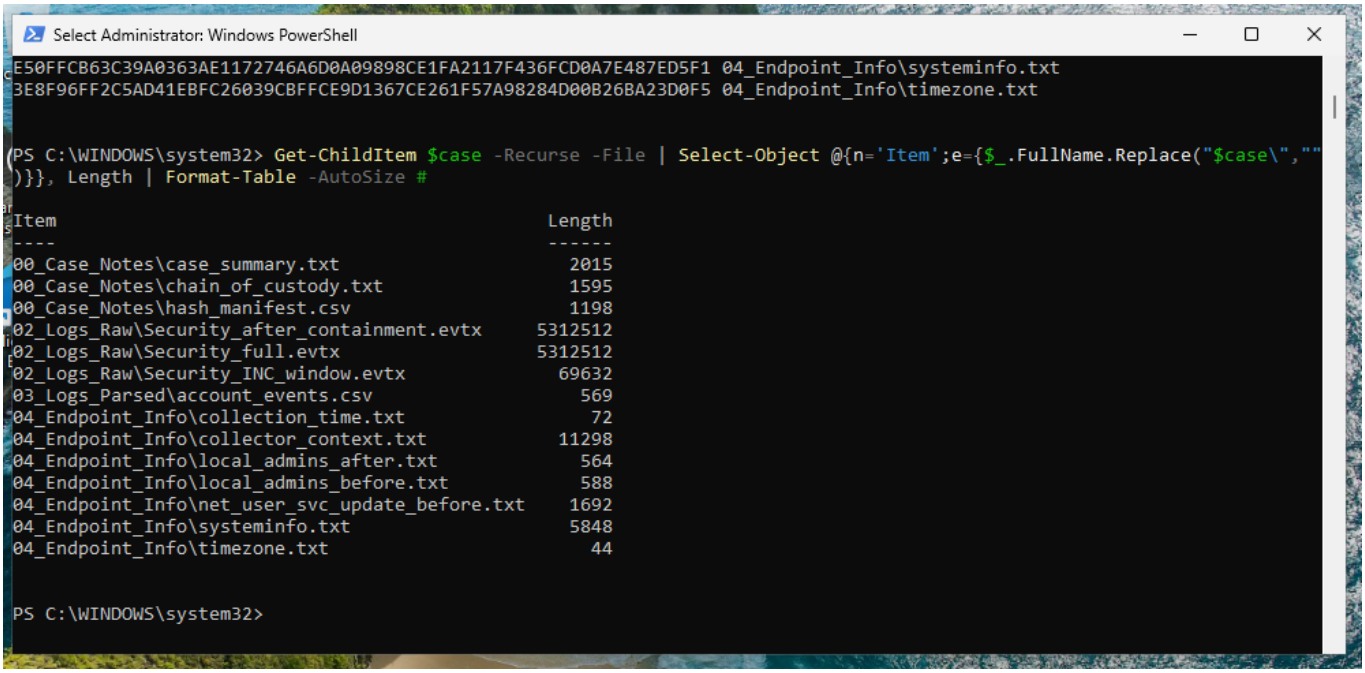


Figure 14. The complete, sealed case folder.

What this screenshot shows:

- 14 files in total: 3 in 00_Case_Notes (summary, chain of custody, hash manifest) and the 11 hashed evidence files.
- Security_full.evtx and Security_after_containment.evtx are both 5,312,512 bytes; Security_INC_window.evtx is 69,632 bytes.
- local_admins_before.txt (588 bytes) is larger than local_admins_after.txt (564 bytes), consistent with one member having been removed.

11. Timeline

A timeline places every event of the case in order, from the start of the actor's session to the sealing of the evidence, each with its time and supporting evidence. It matters because the order of events is what turns separate log entries into an explanation: it shows that the account was created before it was made an administrator, how long it existed, and that its state was captured before it was removed. It also lets a reviewer measure how quickly the activity was found and contained.

All times are IST on 2 October 2026:

Time	Event	Evidence
11:34:36	CyberAmy interactive console logon; session 0x17b4f begins	Figure 8
11:52	Case folder created; audit policy confirmed	Figure 1
12:00:25	Simulation window opens	Figure 2
12:01:54	svc_update created, enabled, password set (4720, 4722, 4724, 4738); added to None (4728) and Users (4732)	Figures 3, 4, 7
12:02:50	svc_update added to Administrators (4732)	Figures 5, 7
12:03:20	Simulation window closes	Figure 2
After 12:03	Detection and triage: initial severity High	Section 5
After 12:08:48	Logs exported and investigated (the export contains an event at 12:08:48)	Figures 6 to 8
Before 12:59	Account and group state captured	Figures 9, 10
12:59:41	svc_update disabled (4725)	Figure 12
12:59:55	svc_update removed from groups and deleted (4733 x2, 4729, 4726)	Figure 12
13:00:09	Removal verified	Figure 11
After 13:00	Post-containment log exported; case notes added; 11 files hashed	Figures 12 to 14

Where an exact time was not recorded, the timeline gives the order of events instead of an invented time.

12. Severity, Verdict and Recommendation

This section is the outcome of the investigation: how serious the activity was, what it turned out to be, and what should happen to the case. It is the part of the report a manager acts on, so each decision is stated plainly and supported by the evidence in the earlier sections.

12.1 Severity

Severity rates how serious an alert is, combining its potential impact with the likelihood that it is malicious. It determines how urgently a case is handled and who must be informed. Severity is assessed twice. The initial rating is made at triage, when little is known and the worst reasonable case must be assumed. The final rating is made after investigation and containment, when the evidence shows what actually happened. Recording both demonstrates that the alert was taken seriously and that the lower final rating was reached through evidence, not assumption.

Stage	Severity	Reason
Initial (at triage)	High	New account given administrator rights within a minute; service-style name; full control of the endpoint possible
Final (after investigation and containment)	Low	Single known actor in a local console session; account never used; account removed; activity authorized

12.2 Verdict

The verdict is the analyst's conclusion about what the activity was. SOC verdicts generally fall into four kinds: a true positive (the detection was right and the activity was malicious), a benign true positive (the detection was right but the activity was authorized or harmless), a false positive (the detection fired although the activity it describes did not occur), and a case that cannot be determined and must be escalated. Choosing correctly matters because it decides the response and, over time, shows which detections are working well.

Benign true positive. The detection was correct, because an administrator account really was created, but the activity was an authorized simulation performed by the account owner, the new account was never used, and it has been removed.

12.3 Closure or Escalation Recommendation

Every investigation ends in one of two ways: the case is closed, or it is escalated to a higher tier such as L2 analysts or the incident response team. A case is closed when the analyst can fully

explain the activity from evidence and no threat remains. It is escalated when something cannot be explained, when there are signs of real compromise, or when the response needed is beyond the analyst's authority. Stating the conditions that would have changed the decision shows that closure was a judgement, not a default.

Recommendation: close the case. No escalation to L2 or incident response is required, because the actor, source and authorization are all established from evidence, the account was contained before any use, and no other asset is involved.

The same evidence would require immediate escalation if any of the following had been true:

- CyberAmy did not recognize the activity, which would mean her administrator account was compromised.
- The session had come from a remote address or a Remote Desktop logon (Logon Type 10).
- svc_update had logged on, or had been used to make further changes.
- Similar accounts had appeared on other endpoints.

In those cases the response would also extend beyond deleting the new account: the CyberAmy password would be reset, and the host would be isolated and examined, because an attacker able to create one administrator account may have made other changes.

13. Recommendations

Recommendations are the forward-looking part of the report. An investigation that ends only with a verdict leaves the environment exactly as exposed as it was before, so each finding is turned into a change that would prevent a similar incident, detect it sooner, or make it easier to investigate. Each recommendation below comes from a specific finding in this investigation and states the finding, why it matters, and the action to take. A summary table with priorities follows.

Recommendation 1: Alert on new local accounts and additions to Administrators

Finding: the activity was found only because the analyst searched for it; no alert was generated, as no Wazuh manager is deployed. Why it matters: a backdoor administrator account is only dangerous for as long as it goes unnoticed, and here it existed for 58 minutes. Action: deploy the Wazuh manager and enroll this endpoint, then test with a controlled account creation that both the account creation (rule 60109) and the addition to Administrators produce alerts. If the Administrators addition does not alert, write a custom rule for event 4732 on that group.

Recommendation 2: Enable process creation auditing

Finding: no Event ID 4688 was recorded in the incident window (Figure 6), so the log shows that the account was created but not which program or command created it. Why it matters: MITRE's detection analytic for T1136.001 relies on linking the account creation event to the process that caused it, such as net.exe or powershell.exe. Without process events, an analyst cannot tell an administrator's deliberate command from a malicious script. Action: enable the Audit Process Creation policy, which records Event ID 4688 (a new process has been created), and enable the Group Policy setting "Include command line in process creation events". Microsoft notes that command lines may contain sensitive information such as passwords, so access to the Security log should be restricted accordingly.

Recommendation 3: Limit day-to-day use of administrator accounts

Finding: all activity was performed from CyberAmy's elevated session, and CyberAmy is used for routine work on this endpoint. Why it matters: MITRE's mitigation M1026 Privileged Account Management, listed for T1136.001, advises limiting the number of accounts permitted to create other accounts and limiting the use of local administrator accounts for day-to-day operations. If a daily-use account is an administrator, anything that compromises it can create backdoor accounts. Action: use a standard account for routine work and a separate administrator account only when needed.

Recommendation 4: Review local accounts and administrator membership regularly

Finding: before containment, the Administrators group contained three members, one of them unexpected. Why it matters: a known-good baseline (here, Administrator and CyberAmy) makes any addition obvious, even if the original alert is missed. Action: record the expected local accounts and administrators for each endpoint and compare against it on a schedule.

Recommendation 5: Use a separate analyst account for evidence collection

Finding: evidence was collected while logged on as CyberAmy, the account that performed the activity. Why it matters: in a real incident, the actor's account may be compromised, and using it mixes the analyst's actions into the evidence. Action: collect evidence with a dedicated analyst account. This was acceptable here only because the analyst and the account owner are the same person.

Recommendation 6: Forward Security logs to central storage

Finding: until exported, the evidence existed only on the endpoint. Why it matters: an attacker with administrator rights, which is exactly what this scenario creates, can clear local logs. Action: forward Security logs centrally, which a deployed Wazuh manager would provide.

Recommendation 7: Record planned tests in advance

Finding: the benign verdict depended on knowing the activity was an authorized test. Why it matters: a prior record of the host, account and time window lets the SOC verify and close such an alert in minutes. Action: log planned security tests before they start.

Recommendation 8: Transfer scripts as files instead of pasting

Finding: a multi-line command pasted into the VM arrived scrambled (Figure 15). Why it matters: a corrupted command could run with unintended effects on a system holding evidence. Action: transfer longer scripts into the VM as files, or enter commands as single lines, and check each line before running it.

13.1 Recommendations Summary

#	Recommendation	Reason	Priority
1	Alert on new local accounts and additions to Administrators	No alert existed; account lived 58 minutes	High
2	Enable process creation auditing	No 4688 events; the creating command is not visible	High
3	Limit day-to-day use of administrator accounts	Daily-use account can create accounts (MITRE M1026)	Medium
4	Review local accounts and administrators regularly	A baseline makes additions obvious	Medium
5	Use a separate analyst account for collection	Collection used the actor's account	Medium
6	Forward Security logs centrally	Local logs can be cleared by an administrator	Medium
7	Record planned tests in advance	Faster verification and closure	Low
8	Transfer scripts as files instead of pasting	Pasted command arrived scrambled	Low

14. Handling Issues During the Investigation

Three issues arose and are recorded here and, where relevant, in the chain of custody.

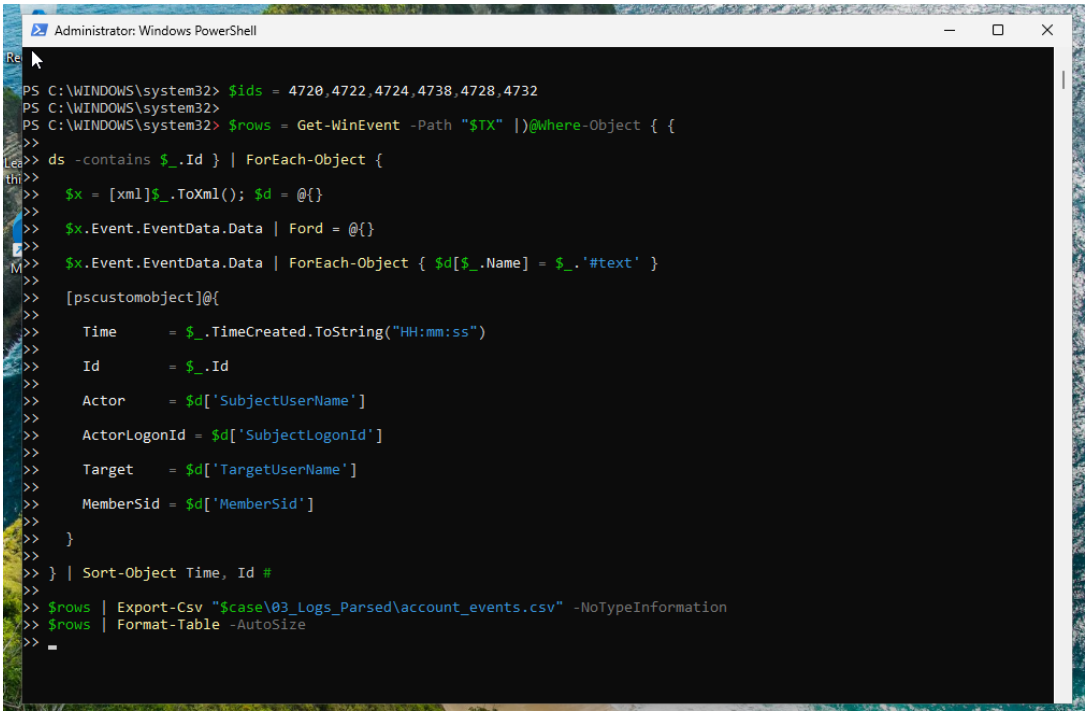


Figure 15. A multi-line command that arrived scrambled when pasted.

What this screenshot shows:

- The third line should begin $rows = Get-WinEvent -Path "$case\02_Logs_Raw\..." but arrived as "$TX" |)@Where-Object { {, and later lines contain text that was never entered, such as Ford = @{}.
- The >> prompts show PowerShell waiting for the rest of an incomplete command. Nothing had run.
- The command was cancelled with Ctrl+C and re-entered as the single lines shown in Step 8, which produced Figure 7. No file was written by the scrambled command.
- Incomplete first filter: the detection query in Step 4 did not include event 4728, which appeared in Event Viewer (Figure 4). The investigation corrected this by exporting every event in the window.
- Label mismatch: PowerShell's TaskDisplayName column labeled 4732 as User Account Management, while Event Viewer shows Security Group Management. The cause was not determined; the report relies on event IDs and Event Viewer.

15. Conclusion

This investigation took a self-generated scenario through the full SOC workflow. Telemetry was confirmed before it was needed; the pattern of a new account gaining administrator rights was detected and triaged as High; the exported logs established who made the changes, from which session, and that the account was never used; the behavior was mapped to MITRE ATT&CK T1136.001 and T1098.007; the account was disabled and deleted, with the clean-up confirmed in the logs; and eleven evidence files were sealed with SHA-256 hashes.

The case is closed as a benign true positive with a final severity of Low and no escalation. Its most useful outputs are the two visibility gaps it exposed: no alert fired for a new administrator account, and no process creation events were recorded. Closing those gaps is what would turn this manual investigation into a detection that works in real time.

References

- MITRE ATT&CK, T1136.001 Create Account: Local Account: https://attack.mitre.org/techniques/T1136/001/
- MITRE ATT&CK, T1098.007 Additional Local or Domain Groups: https://attack.mitre.org/techniques/T1098/007/
- Microsoft Learn, Audit User Account Management: https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/auditing/audit-user-account-management
- Microsoft Learn, Audit Security Group Management: https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/auditing/audit-security-group-management
- Microsoft Learn, 4624(S) An account was successfully logged on: https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4624
- Microsoft Learn, wevtutil: https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/wevtutil
- Microsoft Learn, Command line process auditing (Event 4688): https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-server-2012-R2-and-2012/dn535776(v=ws.11)
- Ultimate Windows Security, Event ID 4728: https://www.ultimatewindowssecurity.com/securitylog/encyclopedia/event.aspx?eventid=4728
- EventSentry, Security Event ID 5379: https://system32.eventsentry.com/security/event/5379
- Wazuh blog, Detecting Windows persistence techniques with Wazuh: https://wazuh.com/blog/detecting-windows-persistence-techniques-with-wazuh/
- Wazuh issue 9058, Group Modification Rules in Windows Security Ruleset Not Firing: https://github.com/wazuh/wazuh/issues/9058
- NIST SP 800-61 Rev. 3 (April 2025): https://csrc.nist.gov/pubs/sp/800/61/r3/final
- Internal: case INC-20261002-001; Day 25 to Day 29 reports.